# Improving ICU Patient Bed Moves Policies with {Simulation-Optimisation / Reinforcement Learning / Q-Learning / Unsupervised Learning / Approximate Dynamic Programming / Markov Decision Processes}

Geraint I. Palmer-Liyu, Mark Tuson, Virginia Ahedo & Paul R. Harper

April 16, 2026

## 1 Introduction

## 2 Background

**TODO**

Here we need some background on the ICU department at the Heath; background and literature on on ICU bed moves, why we move patients, why it's bad to move patients, etc.

## 3 MDP Formulation

**TODO**

Here I am describing the simplified (trivial?) MDP which finds the policy of which block to place newly arriving patients. This will be expanded to include bed moved once this formulation has been tested and experimented with.

We will consider the bed move policy problem as a Markov Decision Process (MDP), which is a tuple $(S, A, P, R)$, with states $s \in S$, actions $a \in A$, transition probabilities $p_{s,s',a} \in P$, and rewards $r_{s,s',a} \in R$. This is a discrete time MDP, with actions being taken at the discrete events when a new patient is admitted. The approach taken to find the optimal policy will be reinforcement learning, a model-free approach, and so the probabilities $P$ need not be defined, and transitions are taken care of with a simulation model. The following sections describe each component.

### 3.1 States

The ICU ward contains 17 beds, which can be partitioned into 9 blocks, shown in Figure 1. These blocks are defined by resource use: normally each block can be overseen by one full time equivalent nurse. A corridor separates blocks 1, 2, and 3, from blocks 4, 5, and 6. Blocks 7, 8, and 9 represent isolation units, and are separated by physical walls.

The model will consider three types of patient, categorised as green, amber, or red patients:

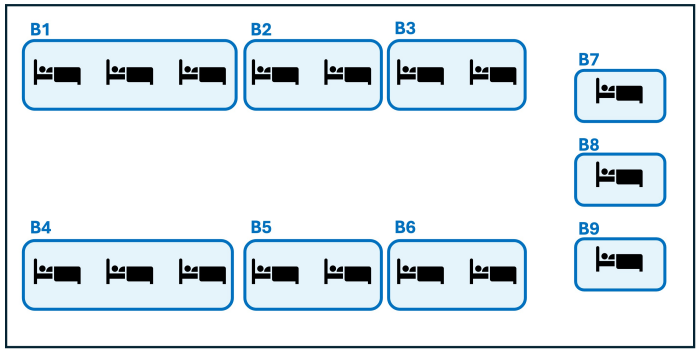


Figure 1: Representation of the 17 ICU beds partitioned into 9 blocks.

- Green: these are the least severe patients. When there are multiple green patients occupying beds in the same block, only one FTE nurse is required for the whole block;
- Amber: these patients require one to one attention from nurses, they will always require their own FTE nurse whichever block they are in, and whichever patients are sharing their block;
- Red: these patients require isolation and one to one attention. If they cannot be places in an isolation unit, they are placed in a shared unit, and will require their own FTE nurse similar to amber patients. They will also incur a penalty cost for each time unit they are placed in a shared block.

Within a block it doesn't make a difference which bed a particular patient is in, as it won't have an effect on resource use or cost. Therefore, the state of the ward can be defined only by how many of each category of patient are in each block. Define the ward state by a $3 \times 9$ matrix $M$, with integer entries $M_{ij}$ corresponding to the number of category $j$ patients (where $j = 0$ corresponds to green patients, $j = 1$ to amber patients, and $j = 2$ to red patients) currently occupying a bed in block $i$.

As an example, consider the ICU state given in Figure 2, where green, amber and red beds indicate beds occupied by green, amber and red patients respectively, and black beds are unoccupied. This corresponds to the ward state:

$$M^\star = \begin{pmatrix} 0 & 2 & 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix} \tag{1}$$

Ward state changes are caused by newly arriving patients of different categories being placed into beds in specific blocks, patient discharges, and moving a patient from one block to another.

**TODO**

Enumerate combinatorially how many valid ward states exist.

Actions only take place when a new patient arrives to the ward, and this patient can be either red, amber of green (category 0, 1, or 2). Therefore the states of the MDP also need to contain information about the newly arriving patient. Let

$$s = (M, p)$$

l

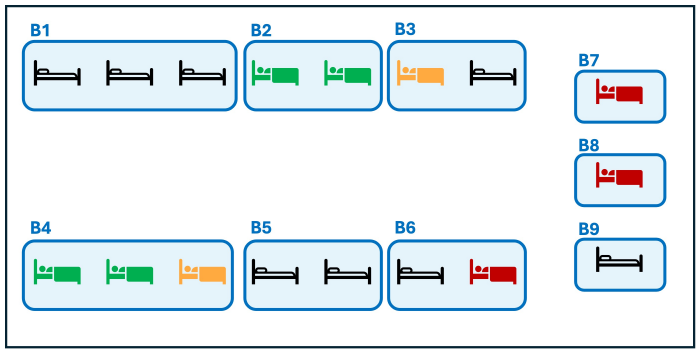


Figure 2: An example ICU state, corresponding to matrix $M^\star$.

represent the state of the MDP, where $M$ is a $3 \times 9$ matrix representing the ward state, and $p \in \{0, 1, 2\}$ indicate the category of the arriving patient.

**TODO**

Enumerate combinatorially how many valid MDP states exist.

## 3.2 Actions

An action in this case is placing a newly arriving patient into an unoccupied bed. As it does not matter which bed within a block to place the patient, the action is the block in which to place the patient. Therefore the maximal set of actions is $A = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Some actions may not be available in all states, for example if block 3 if full, then a patient cannot be places in a bed in block 3. The set of actions available when in a particular state $s$, denoted by $A_s$, will depend on the ward state component, $M$. It will be the the set of blocks that are not full, that is:

$$A_s = \left\{ j \in A \mid \left( \sum_{i=1}^{3} M_{i,j} \right) - K_j < 0 \right\} \tag{2}$$

where $K_j$ is the capacity of each block, that is $K = (3, 2, 2, 3, 2, 2, 1, 1, 1)$.

**TODO**

Enumerate combinatorially how many valid MDP states-action pairs exist.

## 3.3 Rewards

The reward received for transitioning from state $s_1$ to state $s_2$ given action $a$ was taken, $R(s_1, s_2, a)$, consists of three components:

$$\mathrm{R}(s_1, s_2, a) = -(C + P)t_{s_1, s_2} \tag{3}$$

Where

- $t_{s_1,s_2}$ is the time spent in state $s_1$ before transitioning to state $s_2$;
- $C$ is the instantaneous staffing cost, corresponding to the number of staff required to care for the patients per time unit. This can be calculated from the ward-state component of $s_1$, given by Equation 4.
- $P$ is the instantaneous penalty, measures in staff per time unit, corresponding to any additional staffing costs required when patients are not in their preferred blocks, e.g. if a red patient is not in an isolation block. This can be calculated from the ward-state component of $s_1$, given by Equation 5, where $b$ is some penalty parameter.

$$C = \sum_{j=1}^{9} \mathbb{1}_{\{M_{1,j}>0\}} + \sum_{i=2}^{3} \sum_{j=1}^{9} M_{i,j} \tag{4}$$

$$P = b \sum_{j=1}^{6} M_{3,j} \tag{5}$$

### 3.4 Simulation Model

The simulation model is parametrised by:

**TODO**

Parametrise these properly, referring back to Section 2. We can do some data analysis here.

- Green patients arrive according to the Poisson distribution with rate 1.5,
- Amber patients arrive according to the Poisson distribution with rate 1.0,
- Red patients arrive according to the Poisson distribution with rate 0.5,
- Green patients' lengths of stay is Exponentially distributed with rate 0.3,
- Amber patients' lengths of stay is Exponentially distributed with rate 0.7,
- Red patients' lengths of stay is Exponentially distributed with rate 0.4,
- The isolation penalty $b = 8$.

## 4 Reinforcement Learning Framework

Reinforcement learning is a model-free unsupervised learning that allows a central agent to learn optimal policies as it explores the state space during a simulation run. One standard algorithm is Q-learning, with overviews given in [5] and [6]. The idea is that each MDP state and action pair, $(s, a)$ with $s \in S$ and $a \in A_s$, has some value $Q(s, a)$, called $Q$-values, representing the expected long term reward for taking action $a$ when in state $s$, and following an optimal policy from then on. An optimal policy is, for each state, the action that gives the maximum expected reward, given by Equation 6.

$$a^\star = \operatorname*{argmax}_{a \in A_s} Q(s, a) \tag{6}$$

The $Q$-values are found by iteratively updating them, as the simulation visits states, performs actions, and observes some reward, until convergence. The update, implemented once state $s'$ has been reached, after taking action $a$ in state $s$, and observing a reward $R(s, s', a)$ is given in Equation 7.

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(R(s,s',a) + \gamma \max_{a' \in A'_s} Q(s',a')\right) \tag{7}$$

Here $\alpha \in (0,1]$ is the learning rate, a parameter that indicates how important new information is, or how quick the agent is to trust new information; and $\gamma \in [0,1)$ is the discount factor, a parameter that indicates how important future rewards are as opposed to immediate rewards. The ICU ward is stochastic, and so we anticipate a medium to low learning rate $\alpha$ would be required to overcome the variability in the rewards. The aim is for a general state of high reward throughout the ICU's operating times, and so future rewards are just as important as immediate rewards, so we anticipate a high discount factor $\gamma$ would be beneficial. In particular, as the rewards are dependent on the time between updates (Equation 3), immediate rewards are far less meaningful that long run average rewards - for example, one long interval with low instantaneous reward may look better than many shorter periods of high rewards, but it is the long run average that matters.

The reinforcement learning framework used is developed in Python. Due to the very high number of possible states-action pairs, the framework utilises high performance libraries such as Numpy [2] and Numba [4], and was optimised for both speed and memory with help from conversations with Google Gemini [3] to learn and steer some of the software development. It is fully unit-tested and verified, and openly available at `https://github.com/geraintpalmer/bed_moves_policy`.

The $Q$-values are stored in a Numba typed dictionary, an implementation of a hash table, that maps state-action pairs to Q-values, which are generally kept in memory. At the beginning of a run, all state-action pairs are unexplored, and the $Q$-value dictionary is empty. When state-action pairs are visited for the first time, the state-action pair is added to the dictionary, mapping it to the $Q$-value calculated using Equation 7, however this may require the $Q$-value of other unexplored state-action pairs. In these cases, the overall average $Q$-value found up to that point is used as a default value, $\bar{Q}$, which can be obtained by keeping track of the overall average reward, $\bar{R}$, using Equation 4.

$$\bar{Q} = \frac{\bar{R}}{1-\gamma}$$

## 4.1 State Hashing

Due to the vast number of potential state-action pairs, storing full tuples and matrices in the Q-table is very memory intensive. Therefore in order to preserve memory, we define a reversible hash between state-action pairs $(s,a) = ((M,p),a)$ and 64-bit integers (from $-2^{63}$ to $2^{63}-1$), that is a 19-digit integer.

Consider $M$, each entry $M_{ij}$ is an integer in the range $[0, K_j]$, with $K_j \in \{1,2,3\}$, and so groups of columns have a limited set of configurations, which can be bounded. For example, columns 7, 8, 9, representing the isolation wards, and so $K_j = 1$ for all $j \in [7,8,9]$. Therefore there is at most $2^9 = 512$ configurations for these columns (this is an upper bound, as we have ignored the column sum constraint), and so we can assign a 3-digit integer value to the configuration of these three columns. Similar arguments gives that column 1 has at most $4^3 = 64$ configurations, columns 2 and 3 together have at most $3^6 = 729$ configurations, column 4 has at most $4^3 = 64$ configurations, and columns 5 and 6 together have at most $3^6 = 729$ configurations. Using positional concatenation we can therefore transform $M$ into a 13 digit number. Including the arriving patient type $p$ and action $a$ gives us a 15 digit number, well within the confines of 64-bit integers. This is a form of mixed radix encoding. Equation 8 gives this hashing, where $\langle A, B\rangle_F = Tr(A^T B)$ is the Frobenius inner product [1], and the matrix $W$ is given in Equation 9.

$$H((M,p),a)) = 100\langle M, W\rangle_F + 10p + a \tag{8}$$

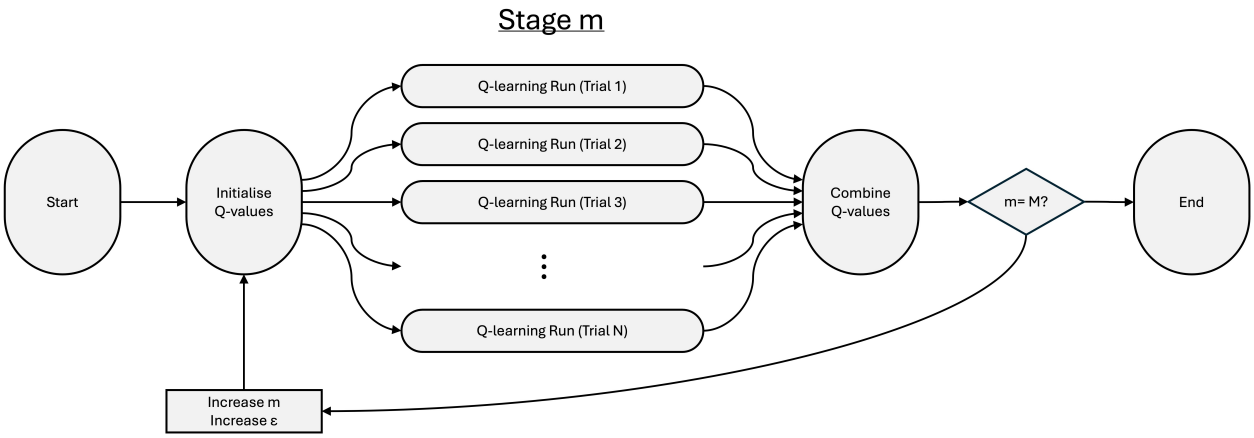


Figure 3: Framework used for training the model.

$$W = \begin{pmatrix} 16 \times 10^{11} & 243 \times 10^{8} & 9 \times 10^{8} & 16 \times 10^{6} & 243 \times 10^{3} & 9 \times 10^{3} & 256 & 32 & 4 \\ 4 \times 10^{11} & 81 \times 10^{8} & 3 \times 10^{8} & 4 \times 10^{6} & 81 \times 10^{3} & 3 \times 10^{3} & 128 & 16 & 2 \\ 1 \times 10^{11} & 27 \times 10^{8} & 1 \times 10^{8} & 1 \times 10^{6} & 27 \times 10^{3} & 1 \times 10^{3} & 64 & 8 & 1 \end{pmatrix} \tag{9}$$

As an example $H((M^\star, 1, 7) = 4893600107217$, where $M^\star$ is the ward state given in Equation 1.

## 4.2 Training Framework

During the simulation's training run, actions are selected using the $\epsilon$-greedy policy [5], that is, for some probability $\epsilon \in [0, 1]$, the agent will choose the best discovered action so far, using Equation 6, with probability $\epsilon$, and choose a random action with probability $1 - \epsilon$. That is, the agent *explores* with probability $1 - \epsilon$, and *exploits* with probability $\epsilon$.

Exploration versus exploitation is an important concept in reinforcement learning, especially in such vast state-spaces. Exploration, or a low $\epsilon$, is important to be able to discover new state-action pairs, however spending too much time exploring can be detrimental, as it wastes time and resources on discovering states that would be seldom visited when following the optimal policy. Exploitation, or a high $\epsilon$, ensures that relevant states are revisited many times, helping to gain enough information on these state-action pairs that their $Q$-values are updated enough times for these to be a good approximation of the expected future rewards. We choose to begin the training run with a low $\epsilon$ and gradually increase is throughout the run - exploring early to find plausible optimal policies, and exploiting later to refine them.

With such a vast state space, and high stochasticity, convergence for all states is practically impossible, and we must settle on very long enough run times to obtain good-enough $Q$-values to read off optimal policies. These long run times are still computationally expensive and time intensive. We therefore train the agent in a parallel framework, visualised in Figure 3.

We train the model over $M$ stages, utilising $N$ parallel workers. At stage $m = 0$, $Q$-value tables are initialised as empty, that is no states-action pairs have been discovered yet, and we set $\epsilon = 0$ for full exploration. These $Q$-value tables are replicated $N$ times, and split over $N$ parallel workers, each running its own training simulation, updating its own table of $Q$-values for a particular stretch of time. Once these parallel training runs are complete, each worker will have its own distinct table of $Q$-values: let $Q_n(s, a)$ represent the $n^{\text{th}}$ worker's estimate of the $Q$-value for the $(s, a)$ state-action pair. Workers also keep track of the number of visits to each state-action pair, let $h_n(s, a)$ be the number of times worker $n$ visited that state-action pair. These tables are then combined, weighting each $Q$-value by the number of visits, that is:

$$Q(s,a) = \frac{\sum_{n=0}^{N} Q_n(s,a) h_n(s,a)}{\sum_{n=0}^{N} h_n(s,a)} \tag{10}$$

In the next state, we update $\epsilon \leftarrow \epsilon + {}^1/_M$, gradually increasing the agents rate of exploitation. Then $Q$-values are initialised with the combined $Q$-value tables found at the end of the last stage, and the training is repeated on parallel workers. This ensures that at the first stage we have full exploration, and by the final stage we have full exploitation, with $\epsilon$ increasing linearly over the stages.

After each stage the combined $Q$-values are saved and so the incumbent policy can be evaluated by running the simulation, without any $Q$-learning, choosing actions based on that incumbent policy.

# 5 Experiments

For a particular run, we evaluate each stage' incumbent policy by running the simulation for 60 trials, for a 30,000 time units, and a warm-up time of 5,000 time units, and recording the overall cost running the ICU for the remaining 25,000 time units when exploiting that incumbent policy. An initial 'random' policy is evaluated for comparison.

First we train the agent using the framework described in Section 4, choosing $\alpha = 0.6$ and $\gamma = 0.6$. During training each stage was run for 500,000 time units, using 20 parallel workers per stage. Figure 4 shows these results. We see that as the stages increase (and hence more training), the agent makes very quick gains in the early stages as the agent explores more, and then slower gains as the agent exploits and refines the policies it has already learned. In this case the agent has made gains from a cost of 366681.25 using the random policy, to 273569.88 using the policy at the end of the 21 stages, an improvement of 25.4%.

As there is such a huge amount of state-action pairs, it is practically impossible to estimate whether the policy has converged or not, though we see any improvements in cost slowing down as the stages increase. We can also count the number of unique state-action pairs visited per stage during the training process, shown in Figure 5. We see some tapering-off of the number of new states-action pairs visited, indicating both a decreasing tendency to explore (from the increasing $\epsilon$), and a gradual saturation of states from previous explorations.

## 5.1 Effect of the learning rate $\alpha$

Using this setup then, we can also look at the effect of the learning rate $\alpha$. Figure 6 shows the percentage improvement in using the incumbent policy per stage for different values of $\alpha$ (left), and the percentage improvement after all 21 stages for different values of $\alpha$ (right).

We immediately see from the left plot that the improvement peaks when $\alpha = 0.6$, indicating that this is the ideal learning rate for this problem, when running under the current setup. However the right plot shows the agent's behaviour as the training progresses: a smaller learning rate results in slower learning (a less steep curve); a high learning rate has faster learning, but cannot smooth out stochasticity of rewards as effectively; while a moderate learning rate can learn quickly and smooth out variability. The speed of learning is important in our case is important: as it would be impractical to run the algorithm until convergence we have to choose some temporal cut-off, and so we want as much learning to occur in the time allotted as possible.

## 5.2 Effect of the discount factor $\gamma$

We can also look at the effect of the discount factor $\gamma$. Figure 7 shows the percentage improvement in using the incumbent policy per stage for different values of $\gamma$ (left), and the percentage improvement after all 21 stages for different values of $\gamma$ (right).

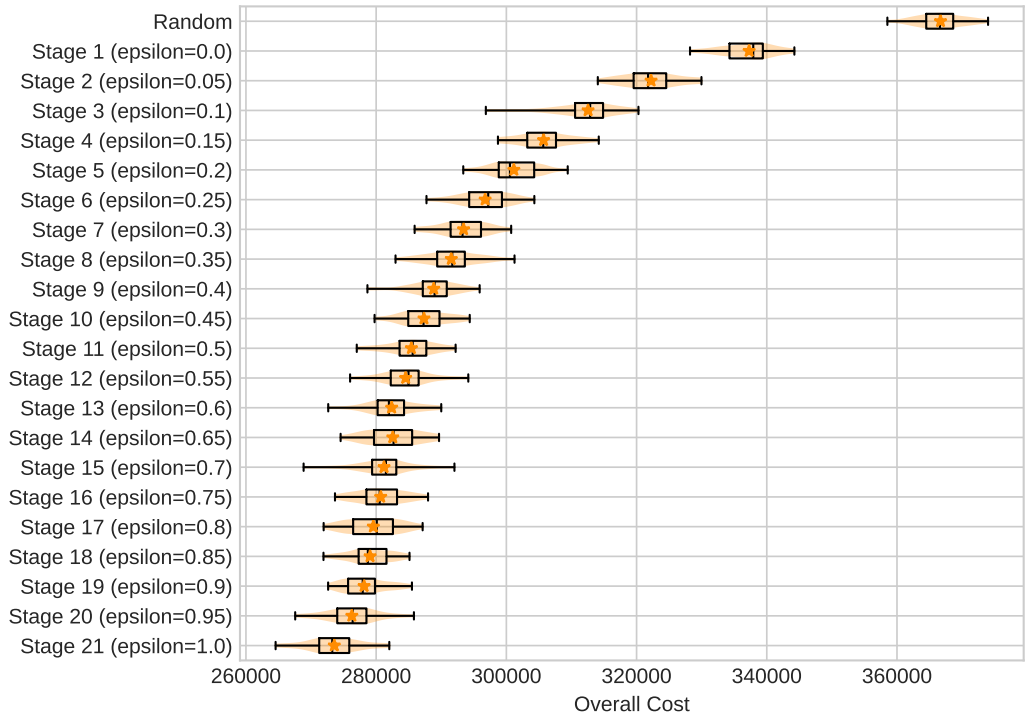


Figure 4: Evaluation of the incumbent policies across the stages.

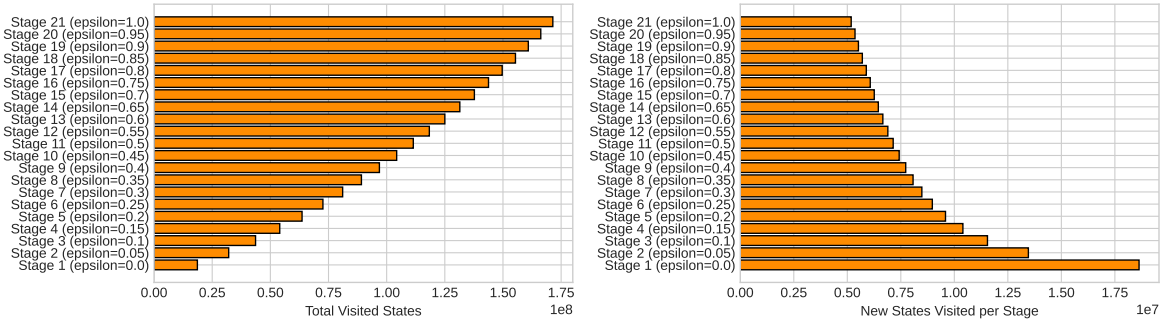


Figure 5: Number of state-action pairs visited in each stage of the training process.

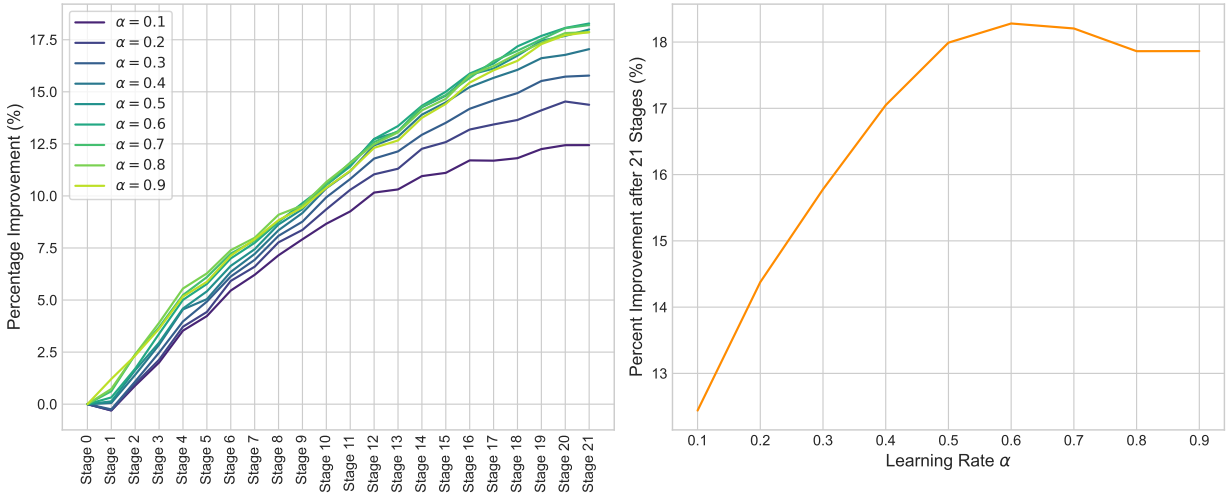


Figure 6: The effect of varying the learning rate $\alpha$.

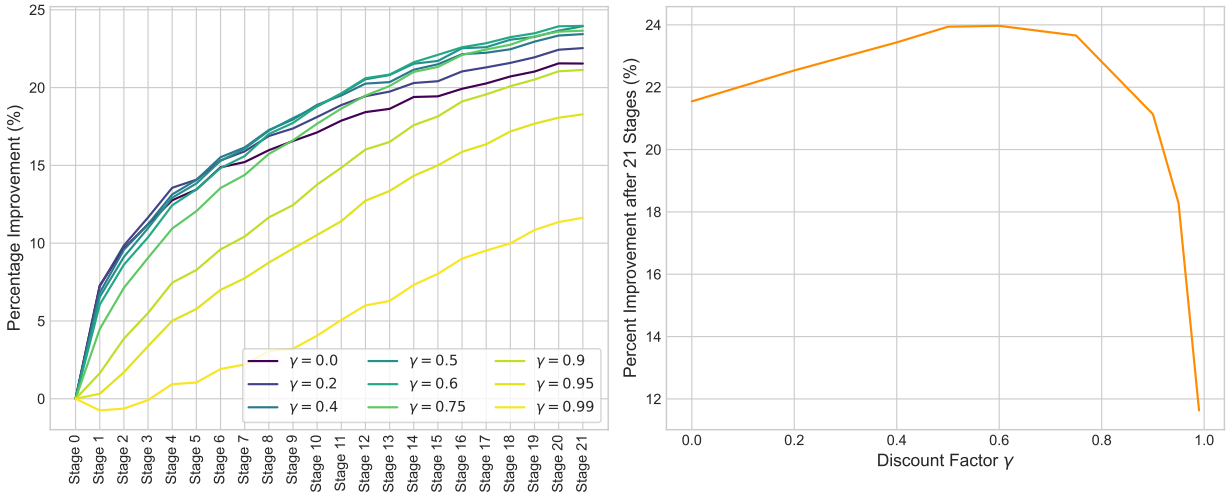


Figure 7: The effect of varying the discount factor $\gamma$.

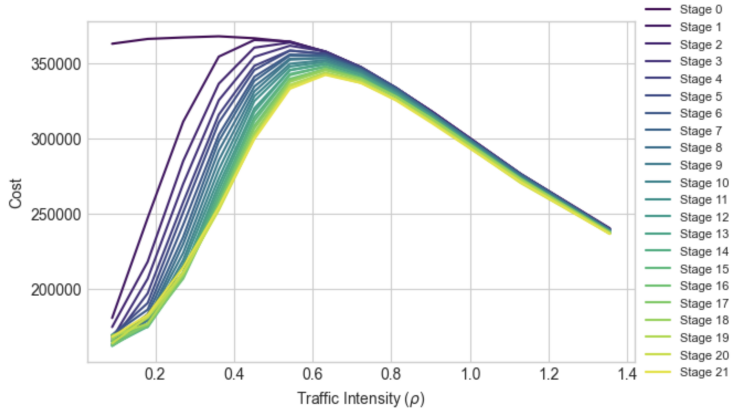


Figure 8: The effect of varying the traffic intensity, $\rho$.

Again we can immediately see from the plot on the left that the improvement peaks when $\gamma$ is around 0.5-0.6. Considering the right plot which shows the agent's behaviour as the training progresses: a high discount factor results in slower learning (a less steep curve), this is due to each update requiring a larger input from future rewards, which themselves need to be well-learned; a small discount factor has faster learning, but account for steady-state or average long-run rewards as effectively. Again, the speed of learning due to the need to choose some arbitrary temporal cut-off.

### 5.3 Effect of the traffic intensity

**TODO**

Results are super preliminary, I am currently re-running these with better parameters.

Bed moves policies might have a greater impact on some ICU wards over other, or there might be some situation where finding optimal bed moves policies can have greater impact. One feature that is likely to have an effect is the ward's business, or the traffic intensity of the ward. To test this, introduce some factor $m$, and multiply all arrival rates by $m$, while keeping the service rates fixed. In this way increasing $m$ increases the traffic intensity $\rho$ of the ward linearly, due to summation of traffic intensities across customer classes:

$$\rho = \rho_{\text{green}} + \rho_{\text{amber}} + \rho_{\text{red}} = \frac{m}{17}\left(\frac{\lambda_{\text{green}}}{\mu_{\text{green}}} + \frac{\lambda_{\text{amber}}}{\mu_{\text{amber}}} + \frac{\lambda_{\text{red}}}{\mu_{\text{red}}}\right) \tag{11}$$

To keep costs and training epochs relatively comparable, we reduce the maximum training and evaluation times by a factor $m$ also. Figure 8 shows the overall evaluation cost, ad $\rho$ varies, for each stage of training.

When $\rho$ is small, we see that there are huge gains to be made, and that most of these gains happen in the early stages, that is after not much training. This implies that these are easy or obvious changes that can be made using human instinct, and a reinforcement learning algorithm may not be needed in practise. The large gains may also show that in such an empty ward, it is easy to reach an 'ideal' scenario, where no penalties or extra staffing is required, as the near empty system is not very complex.

When $\rho$ is very large, there is hardly any gains to be made. This may be because ward is so busy that any bed move actions are not helpful, and penalties and extra staffing are unavoidable whatever moves are taken. Here there is not enough 'wiggle room' to influence much.

This implies that the ideal times to invest in a bed moves policy is for moderate traffic intensities, where $0.3 < \rho < 0.6$. Here large to moderate gains are still possible, however these gains occur in the later stages of training, implying they are less obvious and hence the power of the reinforcement learning algorithm is useful. This is a sweet spot where there is sufficient complexity in the system to require a policy, and enough 'wiggle room' to make substantial gains.

# 6 Human-Readable Policies

TODO

Idea: Currently the policies that come from the reinforcement learning is a mapping from millions / tens of millions of states to optimal actions. I wonder if we can use some sort of machine learning / clustering algorithm to simplify this into general human-readable 'guidelines' rather than a full detailed policy. This simplified policy would of course be less effective than the full policy, but this is something that can be measured and analysed.

# References


[1] Jeff Calder and Peter J Olver. *Linear Algebra, Data Science, and Machine Learning*. Springer, 2025.

[2] Harris CR et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[3] Google DeepMind. Gemini (3 flash), 2026.

[4] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.

[5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.

[6] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.